0004. Median of Two Sorted Arrays

Hard · Binary Search

leetcode.com/problems/median-of-two-sorted-arrays

Problem

- Return the median of two sorted arrays in O(log(m+n)).
- Example: [1,3] and [2] → 2.0; [1,2] and [3,4] → 2.5.

Intuition

- The median splits the merged array into a left half and a right half of equal size.
- Choose i elements from A and j = half - i from B for the left half. The split is correct when A[i-1] <= B[j] and B[j-1] <= A[i].
- Binary search i over the shorter array.

Brute Force

- Merge then pick the middle: O(m+n) time and space.

Optimal Approach

- Ensure A is the shorter array. half = (m + n + 1) / 2.
- Lo = 0, hi = m. i = mid, j = half - i.
- Aleft = i>0 ? A[i-1] : -inf, Aright = i<m ? A[i] : +inf (same for B).
- If Aleft > Bright, hi = i-1; else if Bleft > Aright, lo = i+1; else done.
- Odd total: max(Aleft, Bleft). Even: average of max(lefts) and min(rights).

Worked Example

```
A = [1,2], B = [3,4], half = 2.

lo=0 hi=2 i=1 j=1: Aleft=1 Aright=2 Bleft=3 Bright=4
   Bleft 3 > Aright 2 -> lo=2
lo=2 hi=2 i=2 j=0: Aleft=2 Aright=+inf Bleft=-inf Bright=3  valid
even -> (max(2,-inf) + min(inf,3)) / 2 = 2.5
```

Java Solution

```java
public class MedianOfTwoSortedArrays {
    public double findMedianSortedArrays(int[] a, int[] b) {
        if (a.length > b.length) return findMedianSortedArrays(b, a);
        int m = a.length, n = b.length, half = (m + n + 1) / 2, lo = 0, hi = m;
        while (lo <= hi) {
            int i = (lo + hi) / 2, j = half - i;
            int aL = i > 0 ? a[i - 1] : Integer.MIN_VALUE, aR = i < m ? a[i] : Integer.MAX_VALUE;
            int bL = j > 0 ? b[j - 1] : Integer.MIN_VALUE, bR = j < n ? b[j] : Integer.MAX_VALUE;
            if (aL > bR) hi = i - 1;
            else if (bL > aR) lo = i + 1;
            else {
                if ((m + n) % 2 == 1) return Math.max(aL, bL);
                return (Math.max(aL, bL) + (double) Math.min(aR, bR)) / 2.0;
            }
        }
        throw new IllegalArgumentException("inputs not sorted");
    }
}
```

Complexity

- Time: O(log(min(m, n))). Space: O(1).

Pitfalls

- Searching on the longer array can make j negative.
- Use sentinels for empty sides; one array may be empty.